

Sampling-Based Policy Improvement vs MCTS in Self-Play Chess

An Experimental Guide

TL;DR

We bootstrap a small chess transformer via supervised learning on Stockfish-labeled positions, then run AlphaZero-style self-play training with two competing *policy improvement operators*:

- **Arm A (baseline):** MCTS, implemented via DeepMind’s `mctx`.
- **Arm B (ours):** Importance-weighted Monte Carlo sampling of k -ply trajectories with truncated value-head bootstrap, inspired by *Reasoning with Sampling*.

Both arms share the same network, the same SL bootstrap, and are compared at *equal total network forwards*. Secondary axes: wall-clock time and self-play games count. Primary metric: win rate vs. Stockfish at fixed skill levels.

Compute budget: 600 Colab units (≈ 46 hours A100-equivalent).

Stack: JAX + Flax + `mctx` + `python-chess` + Stockfish.

Honest expectation: MCTS likely wins by some margin. The interesting result is *how much*, and whether sampling closes the gap on wall-clock thanks to embarrassingly parallel rollouts.

Contents

1	Research Question and Framing	3
1.1	The three papers, unified	3
1.2	Sharp research question	3
1.3	Why this is worth running	3
2	Algorithmic Framework	4
2.1	Notation	4
2.2	Arm A: MCTS policy improvement (AlphaZero baseline)	4
2.3	Arm B: Importance-weighted sampling (proposed)	4
2.4	Arm B’: MCMC variant (Phase 4 stretch)	5
2.5	Side-by-side comparison	5
3	Experimental Design	6
3.1	Primary axis: total network forwards	6
3.2	Secondary axes	6
3.3	Pre-registration of hypotheses	6
3.4	Evaluation protocol	6

4	Network Architecture and Bootstrap	7
4.1	Architecture	7
4.2	Bootstrap data: Stockfish labels	7
5	Phased Plan and Compute Budget	8
5.1	Phase 0 – Infrastructure (5 units)	8
5.2	Phase 1 – SL bootstrap (50 units)	8
5.3	Phase 2 – Two-arm self-play training (400 units total)	9
5.4	Phase 3 – Comparison and ablations (100 units)	10
5.5	Phase 4 – MCMC stretch (45 units, only if budget remains)	10
6	Implementation Details	10
6.1	Stack	10
6.2	Sampling arm: vectorization	10
6.3	Stability tricks (apply to both arms)	11
6.4	Practical Colab tips	11
7	Risks and Mitigations	11
7.1	Self-play collapse	11
7.2	Eval noise	11
7.3	Compute overruns	11
7.4	Hidden bugs that look like results	12
8	Reference Hyperparameter Card	12
9	Do / Don't Checklist	13
10	References and Pointers	13

1 Research Question and Framing

1.1 The three papers, unified

The three reference papers can be read as three answers to the same question:

Given a base policy π_θ , how do we obtain a stronger policy π^+ ?

- **Chess Transformer** (Ruoss et al., 2024): use an external oracle (Stockfish) as teacher, train π_θ via supervised learning to imitate it. *Search is amortized into network weights at training time; inference is search-free.*
- **AlphaGo / AlphaZero** (Silver et al., 2016, 2017): use MCTS as a *policy improvement operator*, then distill the improved policy back into π_θ via self-play. Iterate.
- **Reasoning with Sampling** (“*Your Base Model Is Smarter Than You Think*”): leave π_θ untouched. At inference, sample from a sharpened distribution $p_\beta(\tau) \propto \pi_\theta(\tau) \exp(\beta R(\tau))$, e.g. via MCMC, to surface high-reward outputs the base model already places mass on.

This framing suggests a non-obvious combination: **can the sampling operator from the third paper replace MCTS as the policy improvement operator inside the AlphaZero training loop, when the base π_θ is initialized chess-transformer-style?**

1.2 Sharp research question

Given a strong learned policy/value (π_θ, V_θ) pretrained via SL on Stockfish-labeled positions, does importance-weighted Monte Carlo sampling of truncated rollouts, used as the policy improvement operator inside an AlphaZero-style self-play training loop, produce a final policy of strength comparable to MCTS-based AlphaZero, at *equal total inference compute*?

1.3 Why this is worth running

The answer matters either way:

- **Yes (sampling competitive):** the AlphaZero recipe simplifies. No tree code, no UCB, no virtual loss. Embarrassingly parallel inference. Suggests sampling-as-search ideas from the LLM reasoning literature transfer to bandit-like classical RL settings.
- **Sampling close but loses:** you isolate *why* MCTS matters — presumably the tree’s adaptive concentration of compute on promising branches. Quantifying this gap on a strong-prior baseline is itself novel; pre-AlphaGo comparisons used much weaker priors.
- **Sampling significantly worse:** the *Reasoning with Sampling* hypothesis (“base model is smarter than you think; just sample better”) has limits in long-horizon, sparse-reward settings even with strong learned value bootstrapping. That is a real, publishable negative result.

The chess transformer paper appeared in 2024 — the regime of *strong, learned, search-free policy + value* as a starting point for self-play has not been carefully studied. There is genuine room to learn something here.

2 Algorithmic Framework

2.1 Notation

- s : chess position (state).
- a : a move (action) legal at s .
- $\pi_\theta(a \mid s)$: policy network output (categorical over legal moves).
- $V_\theta(s) \in [-1, 1]$: scalar value head, in units of expected game outcome from side-to-move’s perspective.
- $\tau = (s_0, a_0, s_1, a_1, \dots, s_k)$: a trajectory of k plies starting from root s_0 .
- $\pi_\theta(\tau) = \prod_{t=0}^{k-1} \pi_\theta(a_t \mid s_t)$.

2.2 Arm A: MCTS policy improvement (AlphaZero baseline)

At root s , run N_{sim} MCTS simulations using PUCT selection:

$$a^* = \arg \max_a Q(s, a) + c_{\text{puct}} \pi_\theta(a \mid s) \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)}$$

At leaves, expand using π_θ as prior and back up V_θ . After all simulations, the improvement target is

$$\pi_{\text{MCTS}}(a \mid s) = \frac{N(s, a)^{1/\tau}}{\sum_b N(s, b)^{1/\tau}}.$$

Use $\tau = 1$ for the first ~ 30 plies of each self-play game (exploration), $\tau \rightarrow 0$ afterwards. Inject Dirichlet noise at the root: $\pi'(a \mid s) = (1 - \varepsilon) \pi_\theta + \varepsilon \eta$, $\eta \sim \text{Dir}(\alpha)$, with $\alpha = 0.3, \varepsilon = 0.25$.

Implementation: use `mctx.muzero_policy` or `mctx.gumbel_muzero_policy`. Do not write your own MCTS. Reasons: (i) `mctx` is JAX-native and plays well with `vmap`; (ii) it is battle-tested and matches AlphaZero behavior; (iii) your novel work is in Arm B and bugs in the baseline contaminate everything.

2.3 Arm B: Importance-weighted sampling (proposed)

Algorithm 1 Sampling-Based Policy Improvement at root s_0

- 1: **Hyperparameters:** K trajectories, k ply truncation, sharpening β .
- 2: **for** $i = 1, \dots, K$ **do**
- 3: Sample $\tau_i = (a_0^{(i)}, a_1^{(i)}, \dots, a_{k-1}^{(i)})$ from π_θ starting at s_0 .
- 4: If game ends within k plies, set V_i to the terminal value.
- 5: Otherwise, set $V_i = V_\theta(s_k^{(i)})$.
- 6: **end for**
- 7: **Improvement target:**

$$\pi_{\text{sample}}(a \mid s_0) = \frac{\sum_{i: a_0^{(i)}=a} \exp(\beta V_i)}{\sum_{i'} \exp(\beta V_{i'})}.$$

- 8: **return** π_{sample} , and an updated value estimate $V^+(s_0) = \sum_i w_i V_i$ with $w_i \propto \exp(\beta V_i)$.
-

Derivation in one sentence. The target distribution is $p_\beta(\tau) \propto \pi_\theta(\tau) \exp(\beta R(\tau))$ with $R(\tau)$ the (truncated) return. Self-normalized importance sampling from proposal π_θ gives the formula above; the marginal over a_0 is the improved root policy.

Three subtleties to internalize.

1. **Truncation with V_θ is non-negotiable.** Chess games average ~ 80 plies and rewards are sparse. Full rollouts give catastrophic variance. Truncating at k plies and using V_θ for the leaf is exactly the trick AlphaZero uses inside MCTS, and your version inherits it.
2. **β is the sharpening / temperature parameter.** $\beta = 0$ recovers raw π_θ (just frequency counts). $\beta \rightarrow \infty$ gives argmax over best-rollout-value first move. Treat β as a first-class hyperparameter.
3. **Variance of the per-action estimate.** If $K = 100$ and the policy is moderately spread, some legal moves at root may receive 0–2 trajectories, making the per-move estimate noisy. Two mitigations:
 - (i) *Stratified sampling*: sample $\lceil K/|\mathcal{A}(s_0)| \rceil$ trajectories from each legal first move. Reduces variance per move at the cost of ignoring the prior. Recommended for ablation.
 - (ii) *Prior-weighted (default)*: sample first move from $\pi_\theta(\cdot | s_0)$. Higher variance but follows the proposal distribution faithfully (and matches the IS derivation).

2.4 Arm B': MCMC variant (Phase 4 stretch)

The flat IS estimator above is the simplest realization of *Reasoning with Sampling*. The paper itself proposes Metropolis–Hastings sampling from $p_\beta(\tau)$ to better handle high β regimes where IS effective sample size collapses.

Algorithm 2 MH Sampling on Trajectories (Stretch)

- 1: Initialize $\tau^{(0)} \sim \pi_\theta$ (rollout from root for k plies).
 - 2: **for** $t = 0, \dots, T - 1$ **do**
 - 3: Pick a random index $j \in \{0, \dots, k - 1\}$.
 - 4: Propose τ' : keep $a_0, \dots, a_{j-1}$, resample $a'_j, a'_{j+1}, \dots, a'_{k-1}$ from π_θ .
 - 5: Compute acceptance ratio $\alpha = \min \left(1, \frac{\pi_\theta(\tau') \exp(\beta R(\tau'))}{\pi_\theta(\tau^{(t)}) \exp(\beta R(\tau^{(t)}))} \cdot \frac{q(\tau^{(t)} | \tau')}{q(\tau' | \tau^{(t)})} \right)$.
 - 6: With probability α , set $\tau^{(t+1)} = \tau'$; else $\tau^{(t+1)} = \tau^{(t)}$.
 - 7: **end for**
-

The proposal density $q(\tau' | \tau)$ involves the resampled tail under π_θ , which simplifies the acceptance ratio. Mixing time is a real concern for chess trajectory space; run multiple parallel chains and monitor autocorrelation. Do this only after Arm B (flat IS) is working end-to-end.

2.5 Side-by-side comparison

	Arm A: MCTS	Arm B: Sampling
Search structure	Tree (UCB-guided, adaptive)	Flat (independent rollouts)
Compute concentration	On promising branches via UCB	Uniform across sampled trajectories
Parallelization	Needs virtual loss; not embarrassingly parallel	Embarrassingly parallel; full <code>vmap</code>
Inference compute unit	N_{sim} leaf evaluations	$K \times k$ network forwards
Improvement target	Visit-count distribution at root	SNIS-weighted first-move distribution
Critical hyperparams	$N_{\text{sim}}, c_{\text{puct}}, \text{Dirichlet}(\alpha, \varepsilon)$	K, k, β
Failure mode	Exploration collapse without noise	High variance per-move when K small

3 Experimental Design

3.1 Primary axis: total network forwards

Both arms bottleneck on network forward calls. To compare *policy improvement quality*, equalize the per-decision forward count:

$$N_{\text{sim}}^{\text{MCTS}} = K \cdot k \quad (\text{e.g. } 100 \cdot 10 = 1000).$$

A defensible default: 1000 forwards per decision for both arms. Each self-play game has ~ 80 decisions per side $\times 2$ sides $\times 1000$ forwards = 160,000 forwards.

3.2 Secondary axes

- **Wall-clock per decision.** Sampling parallelizes trivially; MCTS does not. On A100 with batched `vmap`, expect Arm B to be 2–4 $\times$ faster wall-clock at equal forwards. *Report this prominently.*
- **Self-play games per Elo gain.** Are improvement targets data-efficient? Track win rate vs. a fixed Stockfish opponent every N training iterations.

3.3 Pre-registration of hypotheses

Before running anything, write these down. Do not modify after seeing results.

- H1** At equal total forwards per decision, MCTS achieves higher win rate vs. Stockfish skill 5 than sampling, by a margin of at least 50 Elo.
- H2** At equal wall-clock per decision, sampling closes at least half the Elo gap from H1.
- H3** Sampling’s gap to MCTS is monotone non-increasing in K (the more trajectories, the better) and non-monotone in β (sweet spot exists).

3.4 Evaluation protocol

- **Opponents:** Stockfish at skill levels $\{0, 3, 5, 7, 10\}$ (approximately Elo 1300 / 1600 / 1900 / 2200 / 2500). Skill cap, not depth cap, to keep eval-time bounded.
- **Games per checkpoint:** 200 games (100 as White, 100 as Black) per opponent skill level. Use a fixed opening book (8-ply) sampled from human master games to ensure variety; both sides play the same 200 openings.
- **Cadence:** evaluate at iteration $\{0, 5, 10, 20, 30, 50\}$.
- **Statistics:** report win rate $\pm 95\%$ CI (Wilson interval). Convert to Elo via standard logistic transform. With 200 games, the CI on Elo is roughly ± 30 at 50% win rate.

Risk note

A 30 Elo CI per evaluation is uncomfortably wide for distinguishing close arms. Mitigations: (i) use BayesElo across the full match graph (every checkpoint vs. every opponent vs. each other); (ii) increase to 400 games per evaluation if budget allows; (iii) re-evaluate the final checkpoint with 4 $\times$ games as the headline number.

4 Network Architecture and Bootstrap

4.1 Architecture

A standard encoder-only transformer with dual heads.

- **Input:** 8×8 board as 64 tokens. Each token embeds (piece type, color) $\in \{\text{empty}, \text{P}, \text{N}, \text{B}, \text{R}, \text{Q}, \text{K}\} \times \{\text{white}, \text{black}\} = 13$ categories. Add learned 2D positional embeddings. Append global tokens for: side-to-move, castling rights (4 bits), en passant target, halfmove clock, fullmove counter.
- **Encoder:** 8 transformer blocks, $d_{\text{model}} = 384$, 6 heads, FFN dim 1536, GELU, RMSNorm. Roughly 7–8M parameters.
- **Policy head:** a learned action embedding for each of 1858 possible moves (the AlphaZero move encoding for chess: from-square $\times$ to-square $\times$ promotion). MLP from per-token-pooled features to logits over moves; mask illegal moves before softmax.
- **Value head:** 2-layer MLP from pooled features to a scalar in $[-1, 1]$ via tanh.

Decision point

Choose now and freeze: dual head (π_θ, V_θ) , encoder-only, ~ 8 M params, 8×8 token grid input. This is the architecture both arms share. Do not modify mid-experiment.

4.2 Bootstrap data: Stockfish labels

Generation pipeline (run on free CPU, not Colab):

1. Source positions: 1–3M games from the Lichess open database, filtered to rating ≥ 2000 for both players. Sample one position per game uniformly to avoid intra-game correlation. Target: ~ 2 M positions.
2. For each position, run Stockfish (via `python-chess.engine`) at depth 12 with `multipv=5`. Record: top-5 moves, their centipawn evaluations.
3. Convert centipawn $\rightarrow V \in [-1, 1]$ via $V = \tanh(\text{cp}/300)$. Convert top- k evals to a soft policy via softmax over V values with temperature 0.1.
4. Store as `.npz` or `tfrecord`; total dataset ~ 5 –10 GB.

SL training:

- Loss: $\mathcal{L} = \mathcal{L}_{\text{policy}} + \lambda_V \mathcal{L}_{\text{value}}$ with $\mathcal{L}_{\text{policy}}$ = cross-entropy against soft policy target, $\mathcal{L}_{\text{value}}$ = MSE against V target, $\lambda_V = 1$.
- Optimizer: AdamW, peak lr 3×10^{-4} , cosine schedule with 1k warmup steps, weight decay 0.01.
- Batch size 1024, 50k–100k steps. ~ 4 –6 hours on A100.
- Target Elo after SL: ~ 2000 against Stockfish skill 0–3, when playing greedily.

5 Phased Plan and Compute Budget

Phase	Goal	Budget	Exit criterion
0	Infrastructure	5 units	Self-play loop runs end-to-end with random net; eval harness reports win rate vs. Stockfish skill 0.
1	SL bootstrap	50 units	Greedy net $\geq 65\%$ win rate vs. Stockfish skill 1 over 200 games.
2A	MCTS arm self-play	200 units	Net trained for 50 self-play iterations, eval at iter 0/5/10/20/30/50.
2B	Sampling arm self-play	200 units	Same as 2A.
3	Comparison & ablations	100 units	Final comparison report; ablations on K, k, β .
4	MCMC stretch	45 units	Only if budget remains and Phase 3 succeeded.

5.1 Phase 0 – Infrastructure (5 units)

What: build the boring scaffolding before touching ML.

- Repo skeleton: `src/{board, net, mcts, sampling, selfplay, train, eval}`.
- `python-chess` wrappers: `position` $\rightarrow$ `tensor`, `move` $\rightarrow$ `index`, batch utilities.
- Stockfish bridge: subprocess pool, non-blocking eval queue.
- Self-play loop with a *random* net (no learning). Sanity check: games complete, no illegal moves, position encoding round-trips.
- Eval harness: orchestrates N games against Stockfish at skill s , returns win/draw/loss, win rate, Elo estimate with CI.
- Logging: `wandb` project. Log per-iteration loss, eval win rates, wall-clock per game, GPU memory.

Compute target: this should run on free-tier CPU mostly; only smoke-test one A100 hour to confirm JAX setup.

5.2 Phase 1 – SL bootstrap (50 units)

What: train the shared base (π_θ, V_θ) .

- Run Stockfish labeling job locally / on free CPU; do not burn Colab on this.
- Train as described in §4.2. Save 3 checkpoints: `base_2M_steps20k`, `base_2M_steps50k`, `base_2M_steps100k`. The middle one is most likely to be your bootstrap checkpoint — the latest one risks overfitting Stockfish’s quirks.
- Eval each greedy ($N = 200$ games) vs. Stockfish skill 0/1/3. Pick the checkpoint with highest aggregated Elo as the shared start of Phase 2.

Decision point

Gate: only proceed to Phase 2 if greedy bootstrap beats Stockfish skill 1 with $\geq 65\%$ win rate. If not, the bootstrap is too weak to detect differences in improvement operators downstream. Fix bootstrap first (more data, longer training, better tokenization).

5.3 Phase 2 – Two-arm self-play training (400 units total)**Self-play loop (each iteration):**

1. Generate 256 self-play games using current net + improvement operator (MCTS for 2A, sampling for 2B). Store $(s, \pi_{\text{improved}}(s), z)$ triples in a replay buffer of size 100k. $z \in \{-1, 0, +1\}$ is the game outcome from the side-to-move’s perspective at s .
2. Train the network for 1000 gradient steps on uniformly sampled batches from the buffer. Loss as in SL but with π_{improved} as policy target and z as value target.
3. Add KL-to-base regularization: $\mathcal{L}_{\text{KL}} = \beta_{\text{reg}} \text{KL}(\pi_{\theta} \parallel \pi_{\theta_{\text{base}}})$, with $\beta_{\text{reg}} = 0.01$. Prevents catastrophic drift in early iterations.
4. Every 5 iterations, run eval ($N = 200$ games per opponent skill).

Hyperparameters (defaults, to be tuned in ablation):

Parameter	Value	Notes
Self-play games / iteration	256	Per arm
Total iterations	50	
Replay buffer size	100,000	Triples
Gradient steps / iteration	1000	
Batch size	1024	
Optimizer	AdamW, lr 1×10^{-4}	With cosine decay
KL regularization β_{reg}	0.01	To frozen Phase-1 base
Dirichlet noise (α, ε)	(0.3, 0.25)	At root, both arms
Arm A (MCTS) specific:		
N_{sim} per move	100	For training; eval may differ
c_{puct}	1.5	
Arm B (Sampling) specific:		
K trajectories	100	Per move
k ply truncation	10	
β (sharpening)	5	Tune in Phase 3
First-move sampling	prior-weighted	Stratified is an ablation

Compute estimate per arm:

- $50 \text{ iter} \times 256 \text{ games/iter} \times 80 \text{ plies/game} \times 1000 \text{ forwards/ply} = 1.0 \times 10^9 \text{ forwards}$.
- At $\sim 0.1\text{ms}$ per forward (batched), ~ 28 hours wall-clock per arm. With `mctx` batching, MCTS arm closer to 35 hours; sampling arm closer to 22 hours.
- In A100 units: ~ 200 units per arm, matching the budget. Tight but feasible.

Risk note

The above is optimistic. Real-world: Stockfish eval queues, JIT recompilation, JAX cold-starts, and Colab disconnects will cost 10–30% overhead. Build automatic checkpointing every iteration. Use Google Drive mount for persistence.

5.4 Phase 3 – Comparison and ablations (100 units)

Headline comparison:

- Each arm’s final checkpoint: 400 games vs. Stockfish skill $\{3, 5, 7, 10\}$.
- Cross-match: each arm’s final checkpoint vs. the other’s, 200 games. Direct head-to-head Elo.
- Wall-clock report: average ms per move on A100 with batched inference at 1000 forwards/move.

Ablations on Arm B (sampling) only:

- $K \in \{25, 50, 100, 200\}$ at fixed $k = 10, \beta = 5$.
- $k \in \{5, 10, 20\}$ at fixed $K = 100, \beta = 5$.
- $\beta \in \{1, 3, 5, 10, 20\}$ at fixed $K = 100, k = 10$.
- Stratified vs. prior-weighted first-move sampling.

For ablations, you do not need full self-play training; just measure win rate vs. Stockfish skill 5 *starting from the Phase 2B final net*, using the ablated improvement operator at *inference only*. This is much cheaper.

5.5 Phase 4 – MCMC stretch (45 units, only if budget remains)

Implement MH sampling per Algorithm 2. Compare to flat IS at high β ($\beta = 20, 50$) where IS effective sample size is expected to collapse. This is the regime where MCMC should help most. If $\beta = 5$ is sweet spot, MH may not buy you anything; that itself is a finding.

6 Implementation Details

6.1 Stack

- **JAX + Flax** for the network. RMSNorm + GELU + standard transformer.
- **mctx** for Arm A. Use `mctx.muzero_policy` with custom `recurrent_fn` that calls your network. See `mctx` examples directory.
- **python-chess** for board logic. Do not roll your own.
- **Stockfish** via `python-chess.engine.SimpleEngine`. Spawn a pool sized to CPU cores.
- **wandb** for tracking.

6.2 Sampling arm: vectorization

The whole Arm B improvement step at root s_0 should be a single `jax.vmap`’d function:

- Input: K replicas of root s_0 , K different PRNG keys.
- Body: for $t = 0, \dots, k - 1$, batch-forward the network on the K current positions, sample K next moves, advance K boards.
- Output: K first moves, K leaf positions, K leaf values.

The board advance step is the only non-JAX-friendly piece (it goes through `python-chess`). One option: pre-compile the legal-move masks and apply moves via a JAX-pure indexing operation, only converting to `python-chess` at game boundaries. Alternative: keep boards on CPU as `python-chess` objects, transfer only the encoded tensor to GPU per step. The second is simpler and probably fast enough at $K = 100$.

6.3 Stability tricks (apply to both arms)

- **Dirichlet noise at root** during self-play. This is the single most important stability fix.
- **KL-to-base regularization** ($\beta_{\text{reg}} = 0.01$). Reference is the frozen Phase-1 checkpoint, not a moving target.
- **Polyak-averaged target net** for value bootstrapping if you find V_θ oscillating. Update as $\theta_{\text{target}} \leftarrow 0.99 \theta_{\text{target}} + 0.01 \theta$.
- **Replay buffer mixing**. Keep some fraction (10%) of SL bootstrap data in the buffer to anchor against distributional drift, especially early.

6.4 Practical Colab tips

- Mount Google Drive at session start; checkpoint every iteration with iteration number in filename.
- Use `!nvidia-smi` on session start; if not A100, save the run for an A100 session.
- Pre-warm JAX JIT: a 5-min compile happens once per session per shape; use stable shapes.
- Track session disconnects via `wandb` alerts. Restart-from-checkpoint should be a one-line operation.

7 Risks and Mitigations

7.1 Self-play collapse

Symptom: after 5–10 iterations, win rate vs. fixed opponent stops climbing or drops; games become repetitive.

Causes & fixes:

- No exploration noise $\rightarrow$ enable Dirichlet, increase ε to 0.4 if needed.
- Replay buffer too small / too on-policy $\rightarrow$ increase to 200k, mix in older data.
- Value head over-confident on rare positions $\rightarrow$ clip targets, use Polyak target net.
- KL regularization too weak $\rightarrow$ raise β_{reg} to 0.05.

7.2 Eval noise

200 games gives Elo CI of ± 30 . To distinguish e.g. 30 vs. 50 Elo gap between arms, you need 800+ games or BayesElo across the full match graph. Plan accordingly.

7.3 Compute overruns

Most common cause: Stockfish labeling underestimated, pulling Phase 1 to twice its budget.

Mitigation: run Stockfish labeling on free CPU *before* touching Colab. Phase 1 budget is for SL training only, not data generation.

Second most common: ablation creep. Before starting Phase 3, write down exactly which ablations you will run. Skip the rest.

7.4 Hidden bugs that look like results

- Forgetting to mask illegal moves in Arm B’s first-move distribution.
- Sign error on V_θ (side-to-move convention) during rollouts that traverse turn changes.
- Off-by-one in k -ply truncation when game ends mid-rollout.
- Treating draw as $z = 0$ in some places and $z = +0.5$ in others.

Prevention: write a unit test for Arm B that, given a π_θ that is uniform over legal moves and a V_θ that is constant 0, returns a uniform improvement target. If it doesn’t, you have a bug.

8 Reference Hyperparameter Card

Network	
Layers	8
d_{model}	384
Heads	6
FFN dim	1536
Activation	GELU
Norm	RMSNorm
Parameters (approx.)	7–8M
SL bootstrap	
Positions	2M (Stockfish depth 12, top-5)
Steps	50k–100k
Batch	1024
Optimizer	AdamW lr 3×10^{-4} , wd 0.01
Schedule	1k warmup, cosine decay
Self-play (both arms)	
Games per iteration	256
Total iterations	50
Replay buffer	100k triples
Grad steps per iteration	1000
Batch	1024
Optimizer	AdamW lr 1×10^{-4}
KL-to-base β_{reg}	0.01
Dirichlet (α, ε)	(0.3, 0.25)
Arm A (MCTS)	
N_{sim} per move	100
c_{puct}	1.5
Implementation	<code>mctx.muzero_policy</code>
Arm B (Sampling)	
K trajectories	100
k ply truncation	10
β sharpening	5
First-move sampling	prior-weighted
Evaluation	
Opponents	Stockfish skill {0, 3, 5, 7, 10}
Games per checkpoint	200 (100 W + 100 B)
Final headline	400+ games at skill 5
Cadence	iter 0, 5, 10, 20, 30, 50

9 Do / Don't Checklist

Do:

- Pre-register hypotheses (§3.3) before touching code.
- Use `mctx` for Arm A. Do not write your own MCTS.
- Generate Stockfish labels on free CPU, not Colab.
- Checkpoint every iteration to Drive.
- Write the unit test in §7.4 before running Phase 2.
- Report wall-clock alongside total-forwards everywhere.

Don't:

- Modify the architecture between Phases 2A and 2B. They must share a base.
- Increase K during training thinking “more is better.” Match the pre-registered budget.
- Cherry-pick the best of multiple seeds without running matched seeds for the other arm.
- Add ablations mid-Phase 3 because they look interesting. Stick to the list.
- Read self-play game logs as evidence of strength; only eval matches count.

10 References and Pointers

- Ruoss et al., *Grandmaster-Level Chess Without Search*, 2024. Code: github.com/google-deepmind/search
- Silver et al., *Mastering the Game of Go with Deep Neural Networks and Tree Search*, Nature 2016 (AlphaGo).
- Silver et al., *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*, 2017 (AlphaZero).
- *Reasoning with Sampling: Your Base Model Is Smarter Than You Think*. Read for the formulation of $p_\beta(\tau)$ and the MH sampler.
- DeepMind `mctx`: github.com/google-deepmind/mctx.
- LeelaChessZero project for engineering reference: lczero.org.

End of guide. Next step: build Phase 0 infrastructure and confirm the eval harness works against Stockfish before writing a single line of training code.